

Practical 9

```
In [5]: import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

# -----
# Hyperparameters
# -----
GAMMA = 0.99
LR_ACTOR = 0.001
LR_CRITIC = 0.005
EPISODES = 1000

# -----
# Environment
# -----
env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

# -----
# Actor Network
# -----
class Actor(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, action_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return F.softmax(self.fc2(x), dim=-1)

# -----
# Critic Network
# -----
class Critic(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

# -----
# Initialize
# -----
actor = Actor()
critic = Critic()
optimizer_actor = optim.Adam(actor.parameters(), lr=LR_ACTOR)
optimizer_critic = optim.Adam(critic.parameters(), lr=LR_CRITIC)

# -----
# Training Loop
# -----
for episode in range(EPISODES):
    state, _ = env.reset()
    state = torch.FloatTensor(state)

    total_reward = 0
    done = False

    while not done:
```

```

# Choose action
probs = actor(state)
dist = torch.distributions.Categorical(probs)
action = dist.sample()

next_state, reward, done, _, _ = env.step(action.item())
next_state = torch.FloatTensor(next_state)
total_reward += reward

# Compute TD Target
value = critic(state)
next_value = critic(next_state)
td_target = reward + (GAMMA * next_value * (1 - int(done)))
advantage = td_target - value

# Actor Loss
actor_loss = -dist.log_prob(action) * advantage.detach()
# Critic Loss
critic_loss = F.mse_loss(value, td_target.detach())

# Update networks
optimizer_actor.zero_grad()
optimizer_critic.zero_grad()
actor_loss.backward()
critic_loss.backward()
optimizer_actor.step()
optimizer_critic.step()

state = next_state

if episode % 50 == 0:
    print(f"Episode {episode}, Total Reward: {total_reward}")

env.close()

```

```

Episode 0, Total Reward: 12.0
Episode 50, Total Reward: 27.0
Episode 100, Total Reward: 51.0
Episode 150, Total Reward: 168.0
Episode 200, Total Reward: 322.0
Episode 250, Total Reward: 73.0
Episode 300, Total Reward: 188.0
Episode 350, Total Reward: 118.0
Episode 400, Total Reward: 125.0
Episode 450, Total Reward: 148.0
Episode 500, Total Reward: 10.0
Episode 550, Total Reward: 10.0
Episode 600, Total Reward: 8.0
Episode 650, Total Reward: 9.0
Episode 700, Total Reward: 9.0
Episode 750, Total Reward: 10.0
Episode 800, Total Reward: 10.0
Episode 850, Total Reward: 8.0
Episode 900, Total Reward: 10.0
Episode 950, Total Reward: 10.0

```